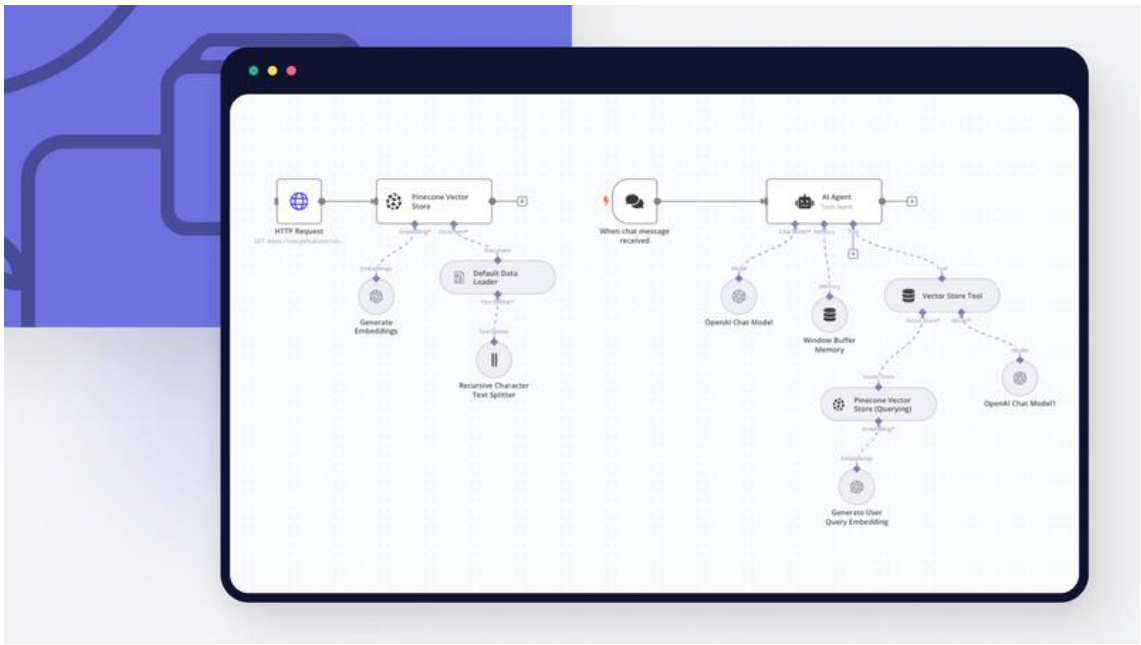
Cree un chatbot RAG de conocimiento personalizado usando n8n

Aprenda a crear potentes chatbots RAG con la automatización visual del flujo de trabajo de n8n. Esta guía paso a paso muestra cómo conectarse a cualquier fuente de conocimiento, indexarla en una base de datos vectorial y crear un chatbot con IA que proporcione respuestas precisas y contextuales.



¿Alguna vez has deseado un chatbot que respondiera preguntas específicas sobre tus datos o documentación, en lugar de simplemente emitir respuestas genéricas? ¡Crea uno que realmente conozca tus datos con Retrieval Augmented Generation (RAG)!

En esta entrada de blog, analizaremos cómo la Generación Aumentada por Recuperación (RAG) permite crear chatbots especializados que superan las limitaciones de las interacciones típicas. En lugar de depender de respuestas genéricas, los chatbots RAG utilizan fuentes de conocimiento externas para ofrecer respuestas precisas e informativas a consultas complejas.

Analizaremos los conceptos básicos de RAG y veremos algunos ejemplos prácticos de flujos de trabajo de RAG. Además, no se trata solo de teoría: demostraremos cómo implementar RAG con n8n, una herramienta de automatización de flujos de trabajo .

¿Qué es el RAG en el chatbot?

La generación aumentada de recuperación (RAG) en los chatbots es una técnica poderosa que combina las fortalezas de los modelos de lenguaje grandes (LLM) con fuentes de conocimiento externas para generar respuestas más relevantes y, lo más importante, precisas.

RAG es particularmente útil en escenarios donde los LLM necesitan acceder y utilizar información que no estaba incluida en sus datos de capacitación inicial, como al responder preguntas sobre dominios específicos, documentos internos o datos que no están disponibles públicamente.

Los chatbots de IA pueden usar RAG para acceder y procesar información de diversas fuentes, incluyendo datos no estructurados como documentos de texto, páginas web y publicaciones en redes sociales, así como datos estructurados como bases de datos y gráficos de conocimiento. Esto les permite:

- proporcionar respuestas más completas e informativas;
- personalizar la experiencia del usuario;
- mantenerse actualizado con la información más reciente;
- Proporcionar respuestas basadas en documentos internos.

Ejemplos de chatbots de RAG

Ahora que hemos explorado los fundamentos de RAG, profundicemos en algunos ejemplos para inspirar tu creatividad.

Demostraremos cómo materializar estas ideas con **n8n** , una potente herramienta de automatización de flujos de trabajo que optimiza la integración de fuentes de conocimiento externas en los flujos de trabajo de RAG. Además, destacaremos por qué n8n es la plataforma ideal para crear chatbots de RAG eficientes, escalables y personalizables.

Los siguientes escenarios son solo un vistazo de lo que es posible cuando se combina el poder de recuperación de la búsqueda semántica con las capacidades generativas de los LLM.

Chatbot de base de conocimientos interna

Un flujo de trabajo que se conecta con los recursos internos de la empresa, centrado en los documentos almacenados en Google Drive. Utiliza un mecanismo para actualizar automáticamente una base de datos vectorial de Pinecone cada vez que se añaden nuevos documentos o se modifican los existentes en las carpetas designadas de Google Drive. Cuando un usuario formula una pregunta, el flujo de trabajo utiliza una combinación de nodos (incluida la búsqueda semántica con Pinecone y un LLM) para recuperar información relevante de los documentos indexados y generar una respuesta.

Un empleado pregunta al chatbot: "¿Cuál es la política de la empresa sobre el teletrabajo?". El chatbot accede al almacén de vectores, recupera los documentos relevantes y genera un resumen de la política.

Chatbot RAG para documentos de la empresa mediante Google Drive y Gemini

Utilice este flujo de trabajo

Documentación de la API del chatbot

Este flujo de trabajo se conecta con la documentación de la API, los ejemplos de código y la documentación del desarrollador. Utiliza un "nodo de función" para analizar las especificaciones de la API y extraer información relevante. Cuando un desarrollador pregunta sobre la API, el flujo de trabajo recupera la documentación y los ejemplos de código pertinentes. Se utiliza un LLM para generar explicaciones concisas, proporcionar ejemplos de uso adaptados al lenguaje de programación del desarrollador e incluso generar fragmentos de código para llamadas comunes a la API.

Un desarrollador pregunta al chatbot: "¿Cómo autentico a un usuario con el protocolo OAuth 2.0 en mi aplicación Node.js?". El chatbot recupera la documentación de la API y los ejemplos de código relevantes, y luego genera un fragmento de código Node.js que muestra el proceso de autenticación, junto con explicaciones y consideraciones de seguridad.

Más adelante en este artículo, repasaremos en detalle los pasos para crear este flujo de trabajo de Chatbot de documentación de API, demostrando cómo conectarse a una especificación de API del mundo real, integrarse con una base de datos vectorial y crear una interfaz de chat utilizando las potentes funciones de n8n.

Chatea con la documentación de la API de GitHub: chatbot impulsado por RAG con Pinecone y OpenAI

Utilice este flujo de trabajo

Chatbot de analista financiero

Este flujo de trabajo se integra con proveedores de datos financieros (p. ej., Bloomberg, Refinitiv). Utiliza nodos de solicitud HTTP para obtener datos de mercado en tiempo real, precios históricos de acciones e informes financieros de empresas. Cuando un usuario realiza una consulta financiera, el flujo de trabajo recupera los datos relevantes y utiliza un LLM para generar análisis exhaustivos, evaluaciones de riesgos o recomendaciones de inversión.

Un analista pregunta al chatbot: "¿Cuál es la percepción actual del mercado respecto a las empresas de energía renovable y cómo se compara con el trimestre anterior?". El chatbot analiza artículos de noticias, la percepción en redes sociales y datos del mercado para ofrecer una respuesta completa, que puede incluir gráficos generados con otros nodos n8n.

Flujo de trabajo RAG con tecnología de IA para el análisis de informes de ganancias de acciones

Utilice este flujo de trabajo

¿Cómo construir un chatbot RAG con n8n?

Pongamos la teoría en práctica creando un chatbot RAG con la potente automatización visual del flujo de trabajo de n8n. Crearemos un chatbot de documentación de API para la API de GitHub, demostrando cómo n8n simplifica la conexión a fuentes de datos, bases de datos vectoriales y LLM. Este ejemplo práctico te guiará paso a paso, desde la extracción e indexación de datos hasta la creación de una interfaz de chat intuitiva.

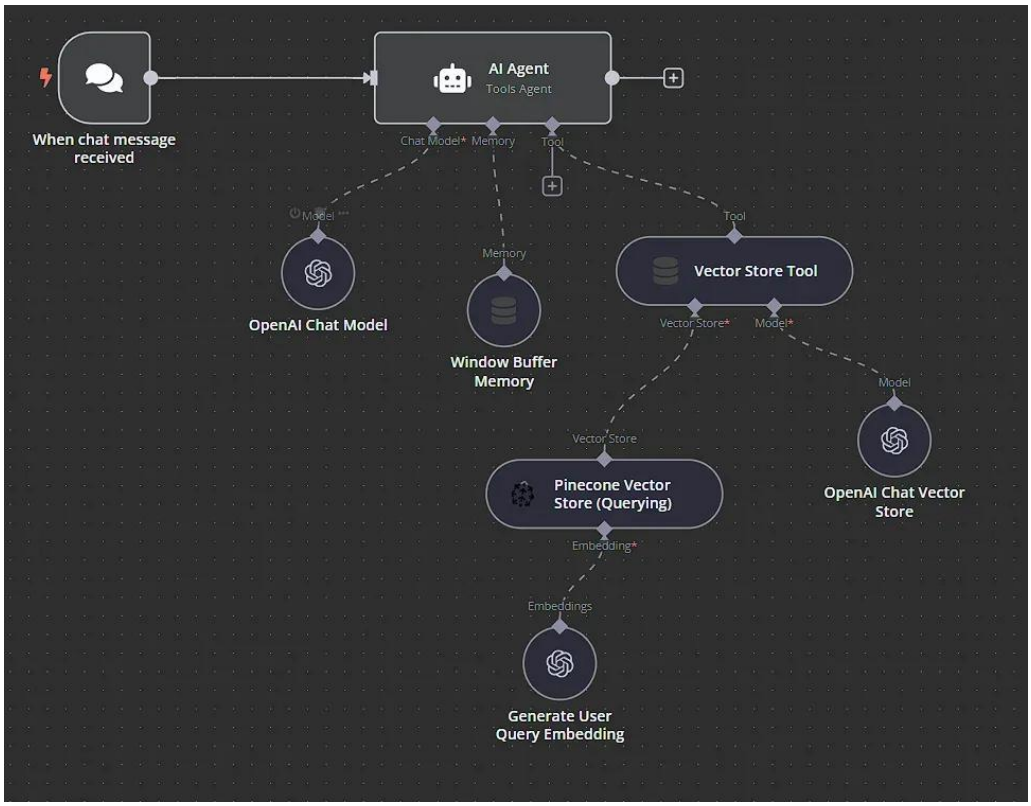
Prerrequisitos

Antes de comenzar a construir, asegúrese de tener la siguiente configuración:

1. **Cuenta n8n:** Necesitará una cuenta n8n para crear y ejecutar flujos de trabajo. Si aún no tiene una, puede crear una cuenta en la nube n8n o alojarla usted mismo .
2. **Cuenta de OpenAI y clave API:** Usaremos los modelos de OpenAI para generar incrustaciones y respuestas. Necesitará una cuenta de OpenAI y una clave API. Puede encontrar esta página de documentación sobre la configuración de credenciales de OpenAI en n8n .
3. **Cuenta de Pinecone y clave API:** Usaremos Pinecone como base de datos vectorial para almacenar y recuperar las incrustaciones de la documentación de la API. Puedes crear una cuenta gratuita en el sitio web de Pinecone y consultar la página de documentación sobre cómo configurar las credenciales de Pinecone en n8n .

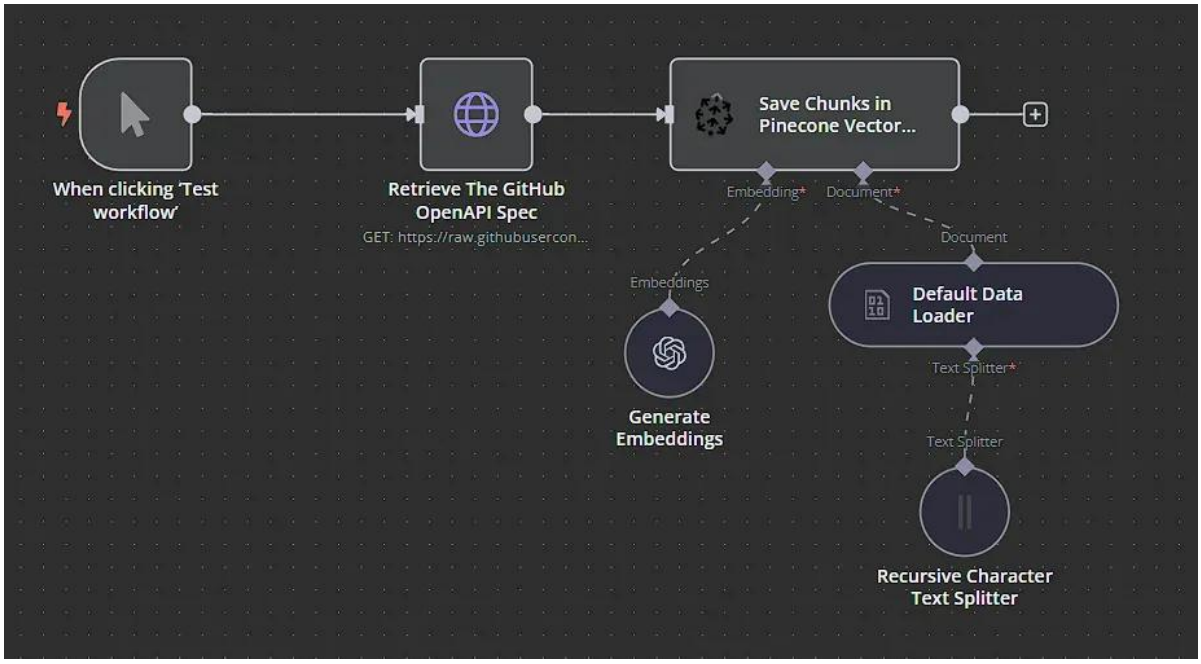
Conocimiento básico de bases de datos vectoriales: Aunque no es estrictamente necesario, comprender los conceptos básicos de las bases de datos vectoriales será útil. Puede consultar la documentación de n8n para obtener información básica: Uso de bases de datos vectoriales en n8n .

Una vez que cumplas con estos requisitos, ¡estarás listo para empezar a crear tu chatbot RAG! Puedes continuar importando el flujo de trabajo:



Paso 1: Configurar la fuente de datos y la extracción de contenido

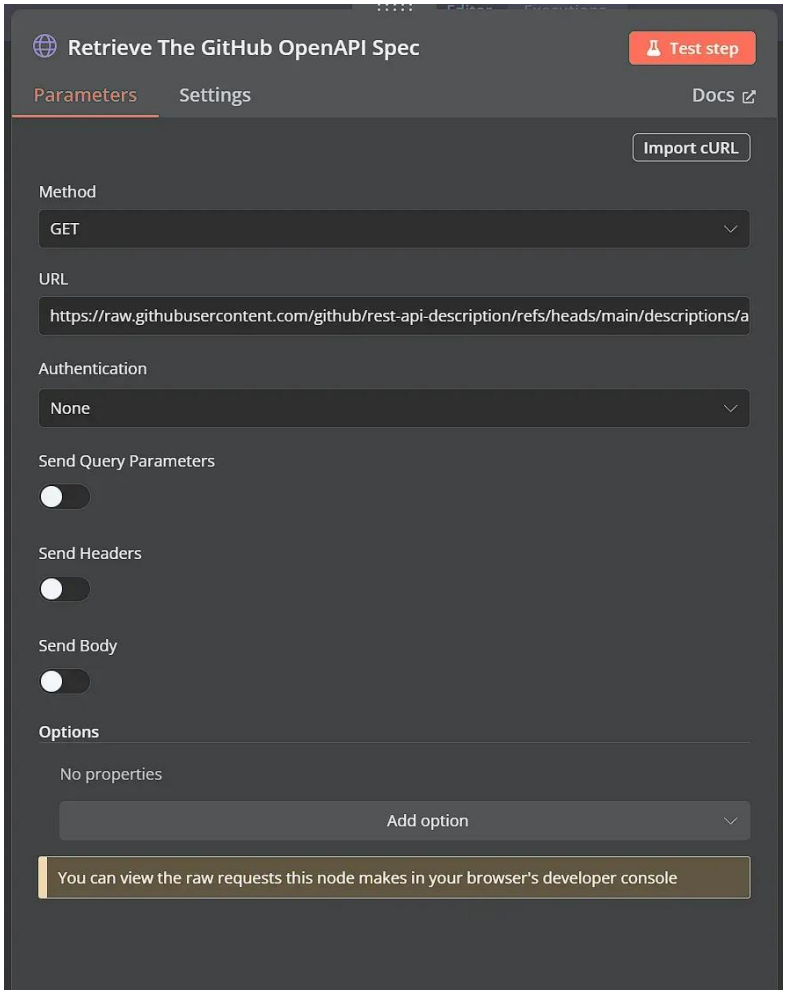
Este flujo de trabajo consta de dos partes. La primera se encarga de obtener los datos e indexarlos en la base de datos vectorial Pinecone, y la segunda se encarga del chatbot de IA.



Cargando datos en el almacén de vectores

Usando el **nodo de solicitud HTTP,** obtenemos la especificación OpenAPI 3.0 de GitHub. Simplemente usamos esta URL sin procesar de GitHub para obtener el

archivo JSON del repositorio. Deje las demás opciones con sus valores predeterminados.

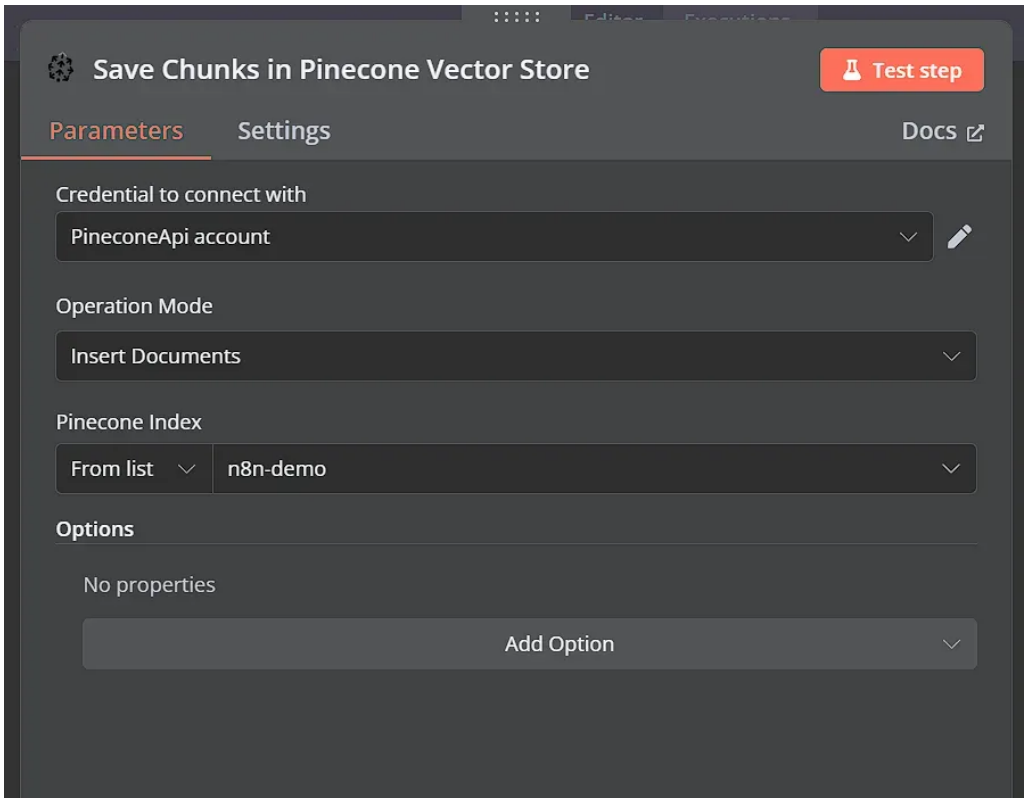


Recuperar la especificación OpenAPI de GitHub

Este nodo realiza una solicitud GET a la URL especificada, que apunta a la representación JSON sin procesar de la especificación de la API de GitHub. La respuesta es el archivo completo de la especificación de OpenAPI. Este archivo se utilizará en el siguiente paso.

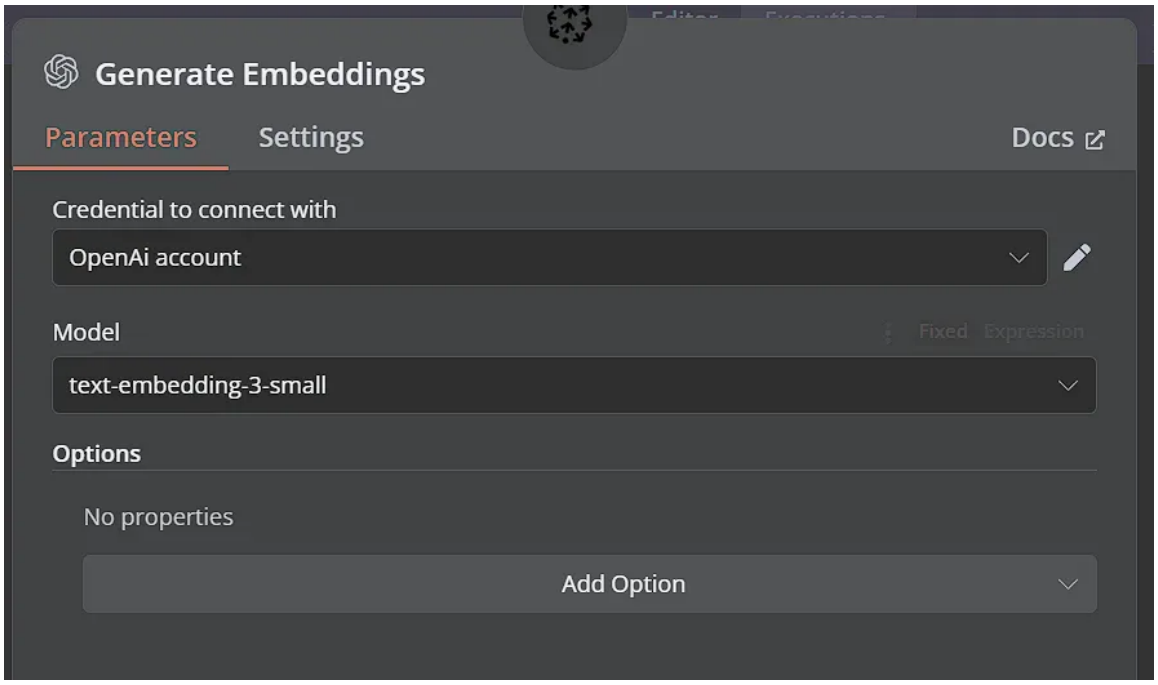
Paso 2: Generar incrustaciones

En este importante paso, transformamos los fragmentos de texto de la documentación de la API en representaciones vectoriales numéricas conocidas como incrustaciones. Estas incrustaciones capturan el significado semántico de cada fragmento de texto, lo que nos permite realizar búsquedas de similitud posteriormente. Podemos hacerlo mediante el nodo Pinecone Vector Store .



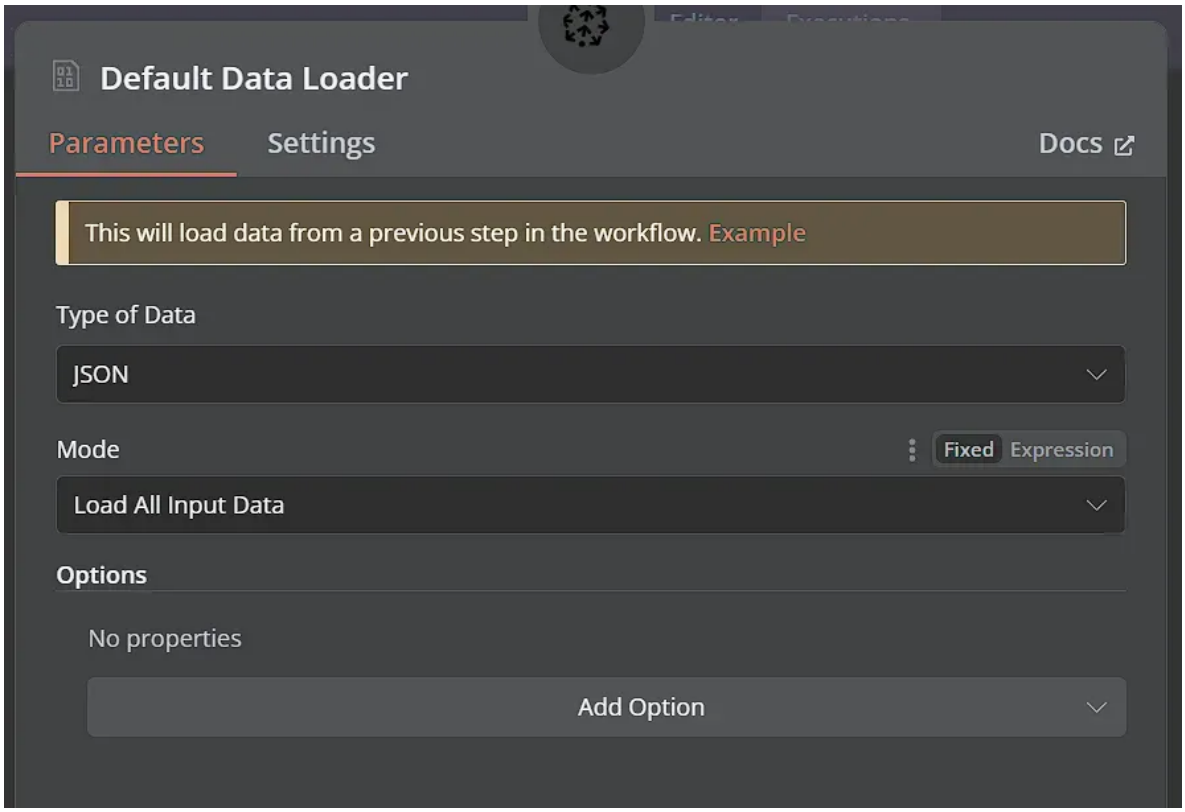
Configurar el guardado de fragmentos de documentos en la tienda de vectores Pinecone

Luego necesitamos conectar un nodo Embeddings OpenAI al nodo Pinecone Vector Store.



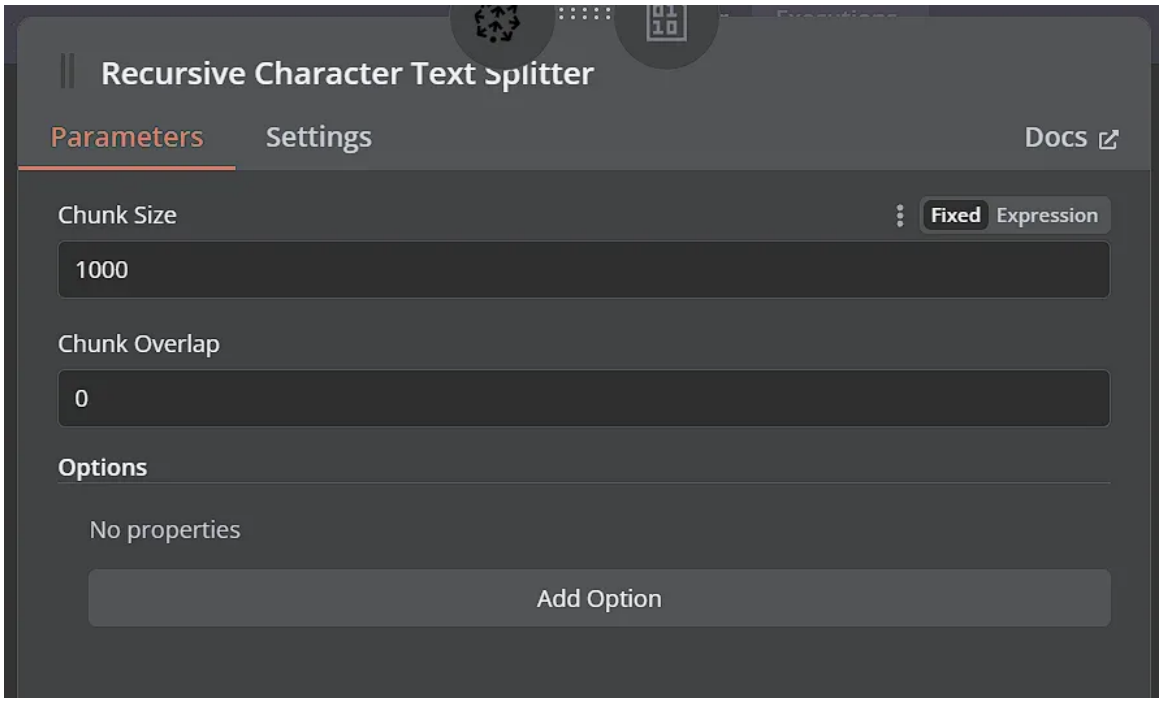
Generar incrustaciones de documentos

Este nodo toma los fragmentos de texto y genera sus incrustaciones vectoriales correspondientes utilizando el modelo de incrustación de OpenAI. Use sus credenciales de OpenAI y seleccione *text-embedding-3-small* como **Modelo** .



Configurar el cargador de datos predeterminado para RAG

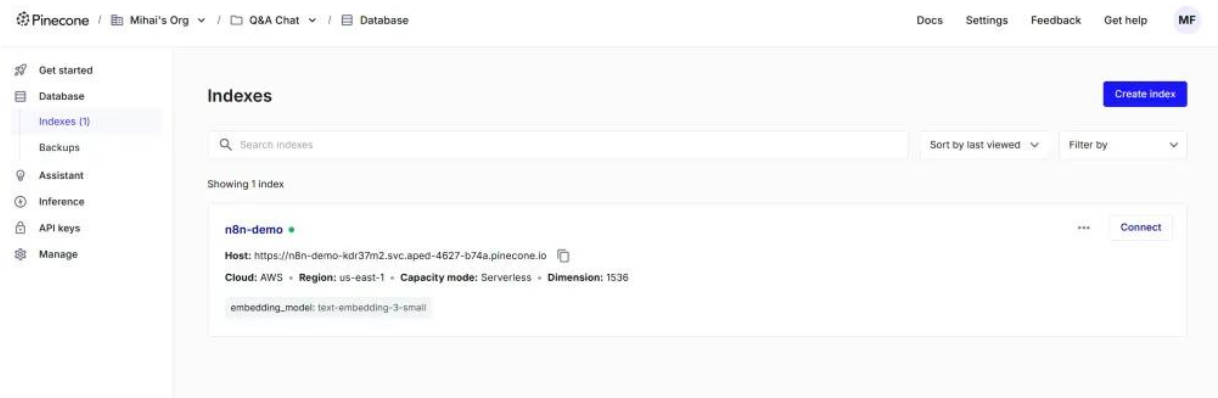
También necesitamos conectar el nodo Cargador de Datos Predeterminado y el nodo Divisor de Texto de Caracteres Recursivos . Puedes dejar todo como predeterminado.



Configurar un divisor de caracteres recursivo

Paso 3: Guarde los documentos y las incrustaciones en la tienda de vectores de Pinecone

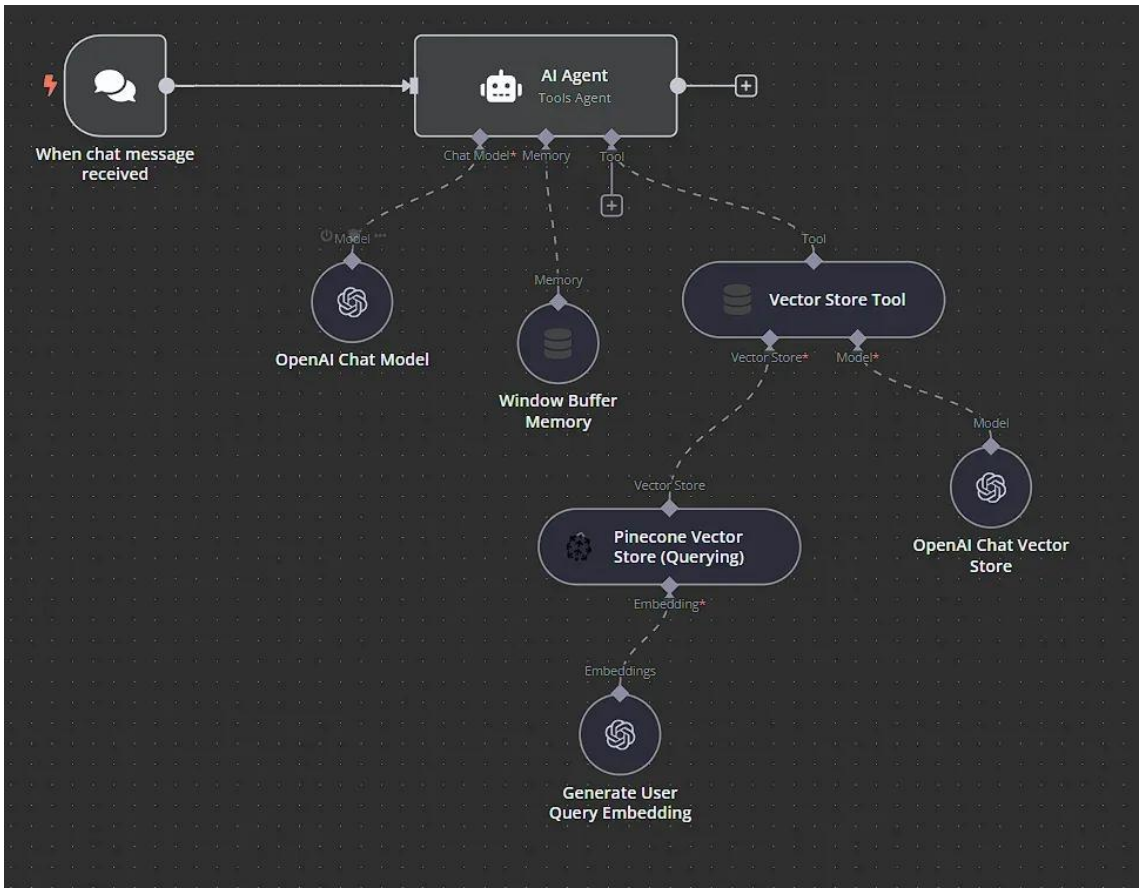
Ahora puede ejecutar esta parte del flujo de trabajo. Generar todas las incrustaciones y guardarlas en Pinecone podría llevar un tiempo, especialmente si el archivo de especificación de la API es grande. Una vez hecho esto, su panel de Pinecone debería mostrar algunos datos en ese almacén de vectores:



Índice de tienda de vectores de piñas

Paso 4: Construir la lógica central del chatbot

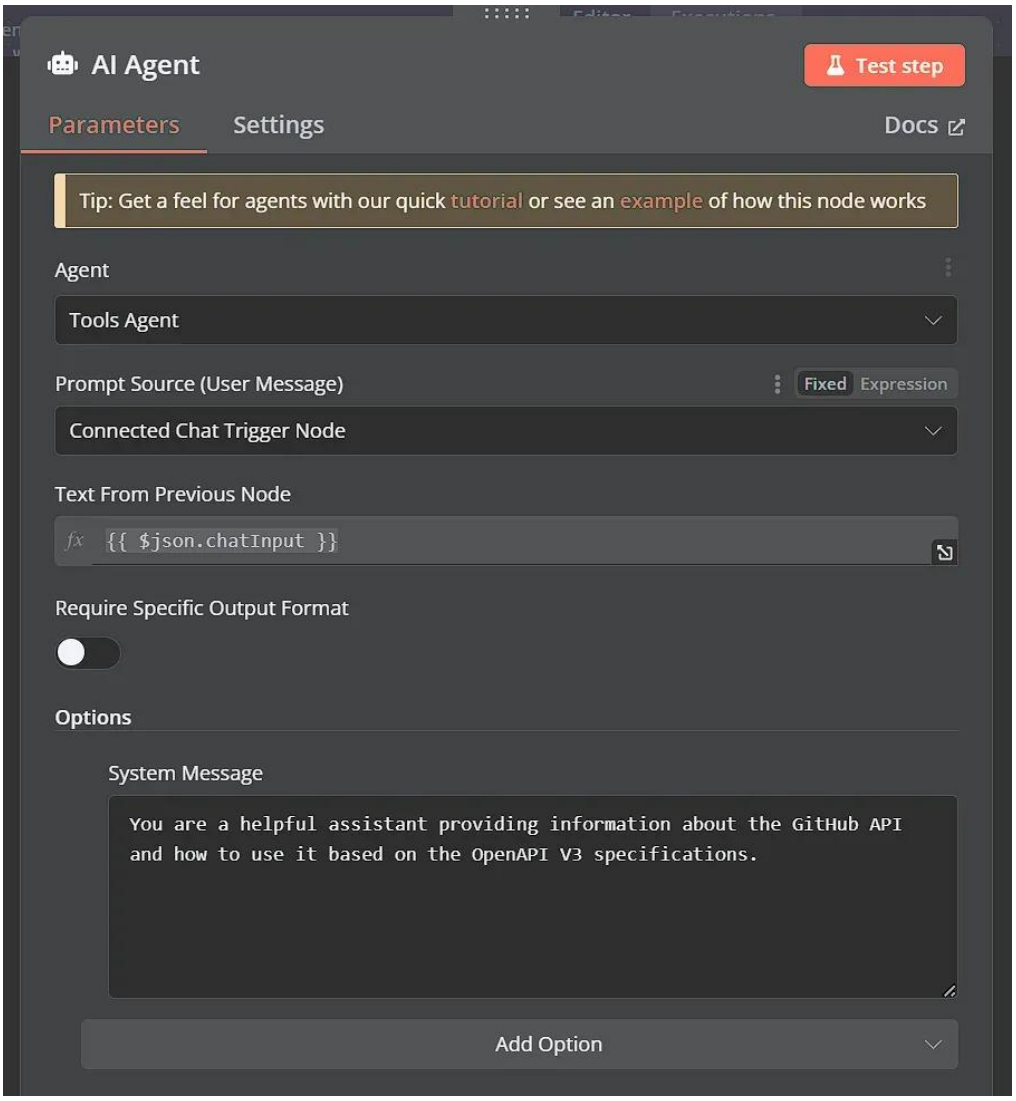
Con nuestra especificación de API indexada en la base de datos vectorial, ahora podemos configurar la parte de consulta y generación de respuestas del flujo de trabajo. Esto implica recibir la consulta del usuario, buscar documentos relevantes en el almacén vectorial y generar una respuesta mediante un LLM.



Flujo de trabajo del chatbot n8n de RAG

Utilice el nodo **Activador de Chat** para recibir la información del usuario. Este nodo actúa como punto de entrada para la interacción del usuario, activando el flujo de trabajo al recibir un nuevo mensaje de chat. Deje la configuración predeterminada por ahora.

Luego, conectamos el nodo Activador de Chat a un nodo Agente de IA . Este nodo organiza los pasos de recuperación y generación. Recibe la pregunta del usuario y los documentos relevantes, y solicita al LLM que genere una respuesta.

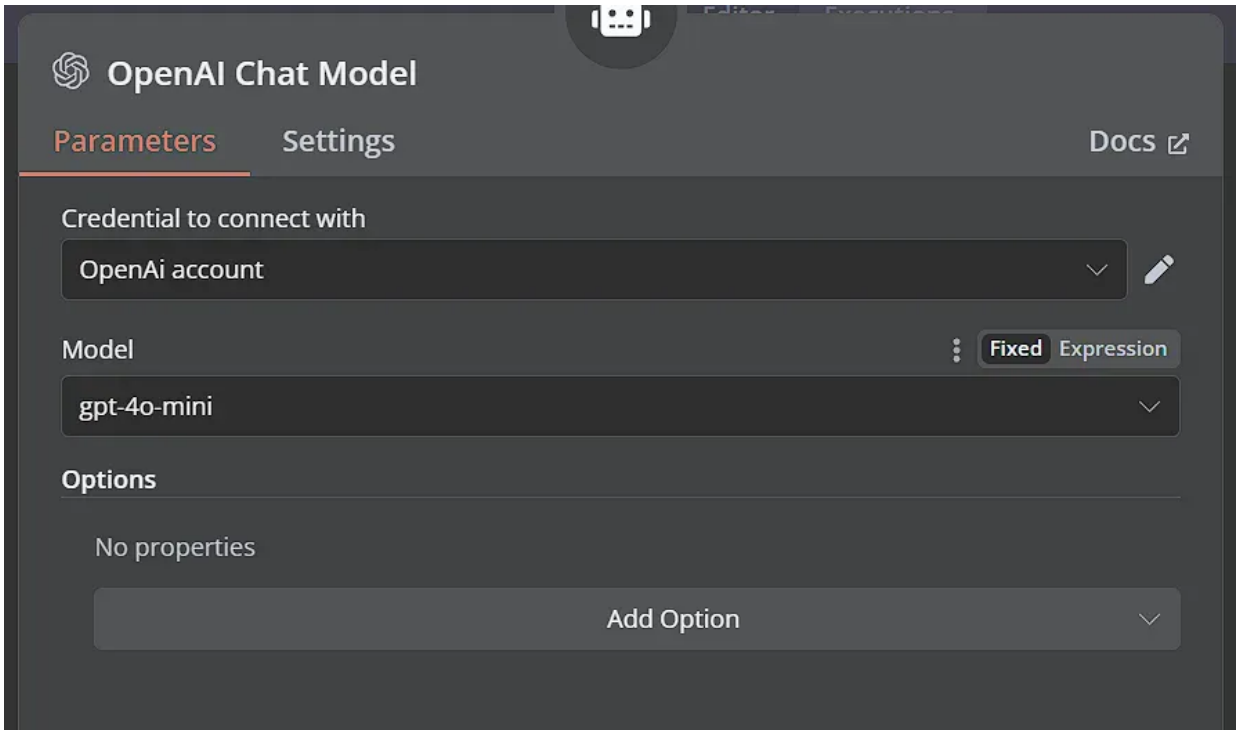


Configurar el nodo del agente de IA n8n

Podemos seleccionar el tipo **de agente** como *Agente de herramientas* . También podemos configurar un *mensaje del sistema* , que se combinará con la entrada del usuario para crear un mensaje para el LLM. A continuación, se muestra un ejemplo sencillo de un mensaje del sistema:

You are a helpful assistant providing information about the GitHub API and how to use it based on the OpenAPI V3 specifications.

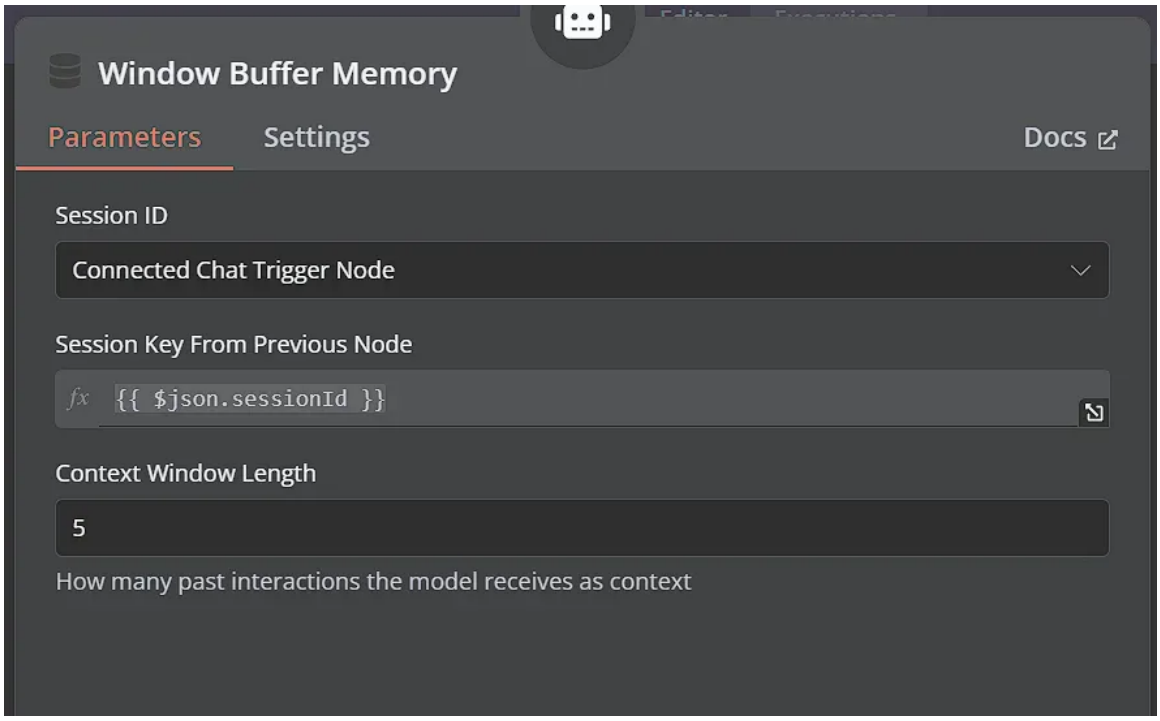
A continuación, conecte el nodo Agente de IA al nodo Modelo de Chat de OpenAI . Este nodo se encarga de procesar la consulta del usuario, junto con los fragmentos de texto recuperados, y de usar un LLM de OpenAI para generar una respuesta final completa.



Configurar OpenAI GPT-4o-mini como el modelo de chat principal

Seleccione sus credenciales de OpenAI y, en el menú desplegable Modelo, elija el modelo eficiente gpt-4o-mini.

Conecte el nodo de memoria de búfer de ventana al nodo de agente de IA. Este nodo proporciona memoria a corto plazo para la conversación, lo que permite al LLM responder preguntas de seguimiento y usar indicaciones y respuestas anteriores para mejorar sus respuestas. Puede dejar la configuración predeterminada.



Implementar la memoria de chat utilizando la memoria del búfer de ventana

Paso 5: Recupere información utilizando la herramienta de almacenamiento de vectores

¡Ahora viene la parte crucial que transforma esto en un chatbot RAG en lugar de un chatbot de IA normal!

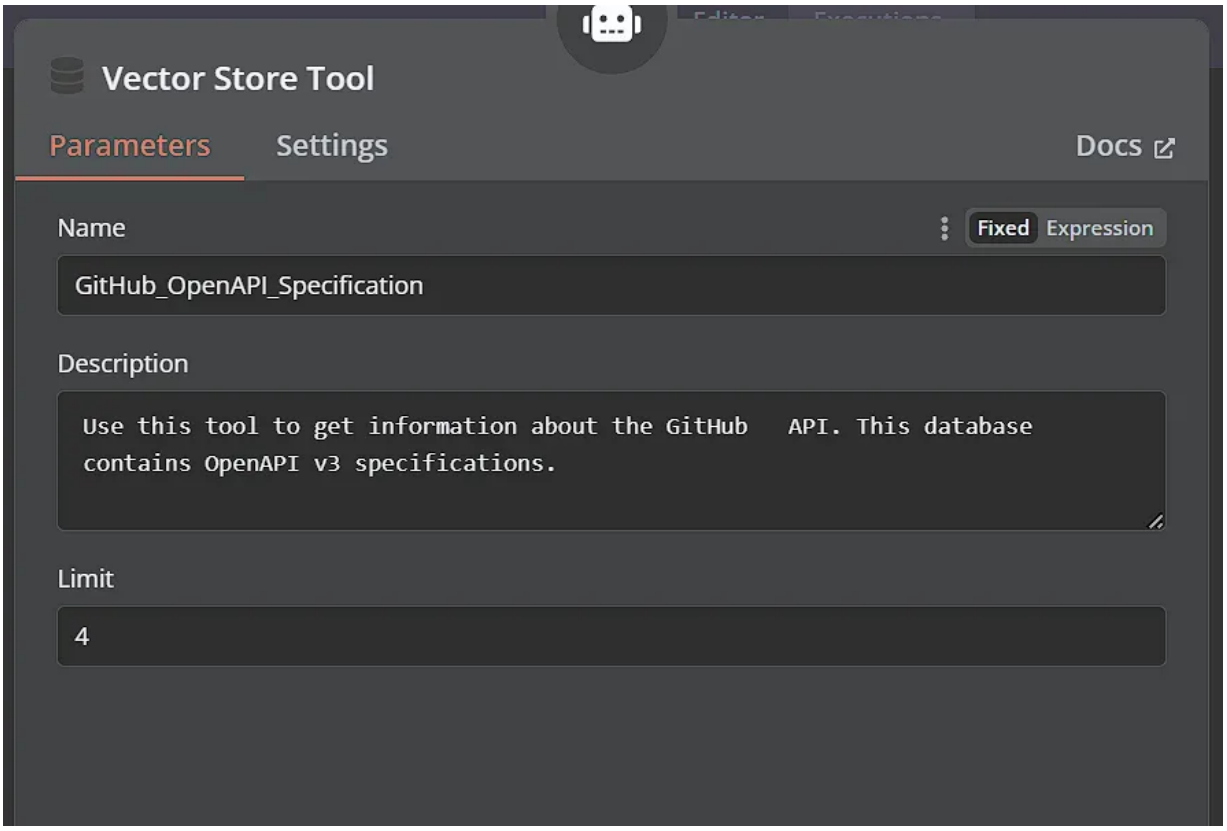
Conecte un nodo de la herramienta Vector Store al nodo **AI Agent** . El nodo utiliza la incrustación de la consulta del usuario para realizar una búsqueda de similitud con las incrustaciones de los fragmentos indexados de la especificación de API.

Configuración de la herramienta de almacenamiento de vectores

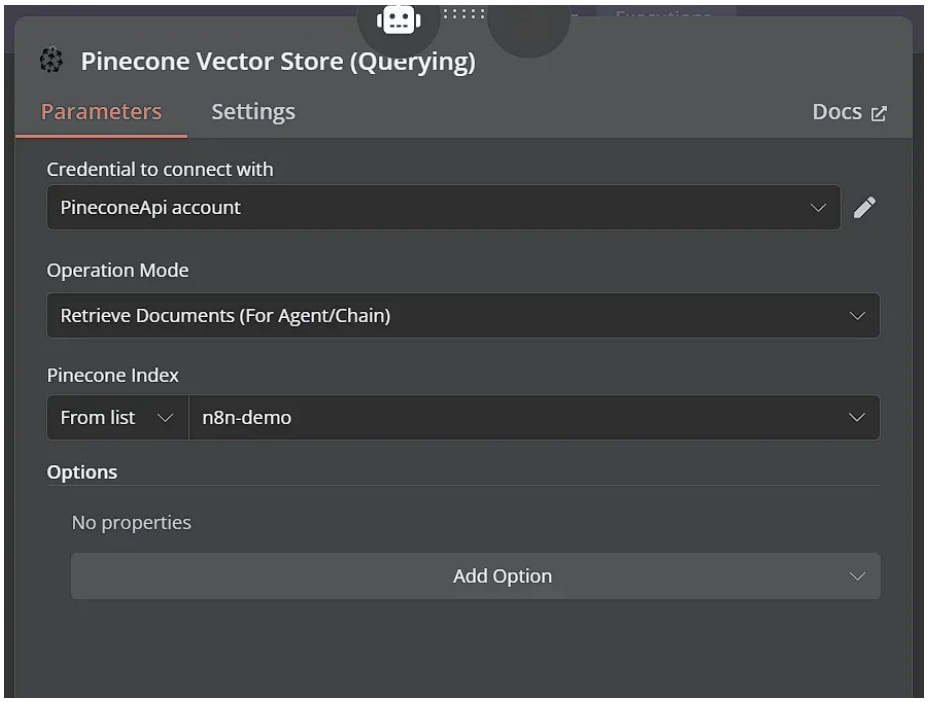
Asigne a esta herramienta un nombre descriptivo y una descripción para que el LLM sepa cuándo usarla. Para este ejemplo, puede usar la siguiente descripción:

Use this tool to get information about the GitHub API. This database contains OpenAPI v3 specifications.

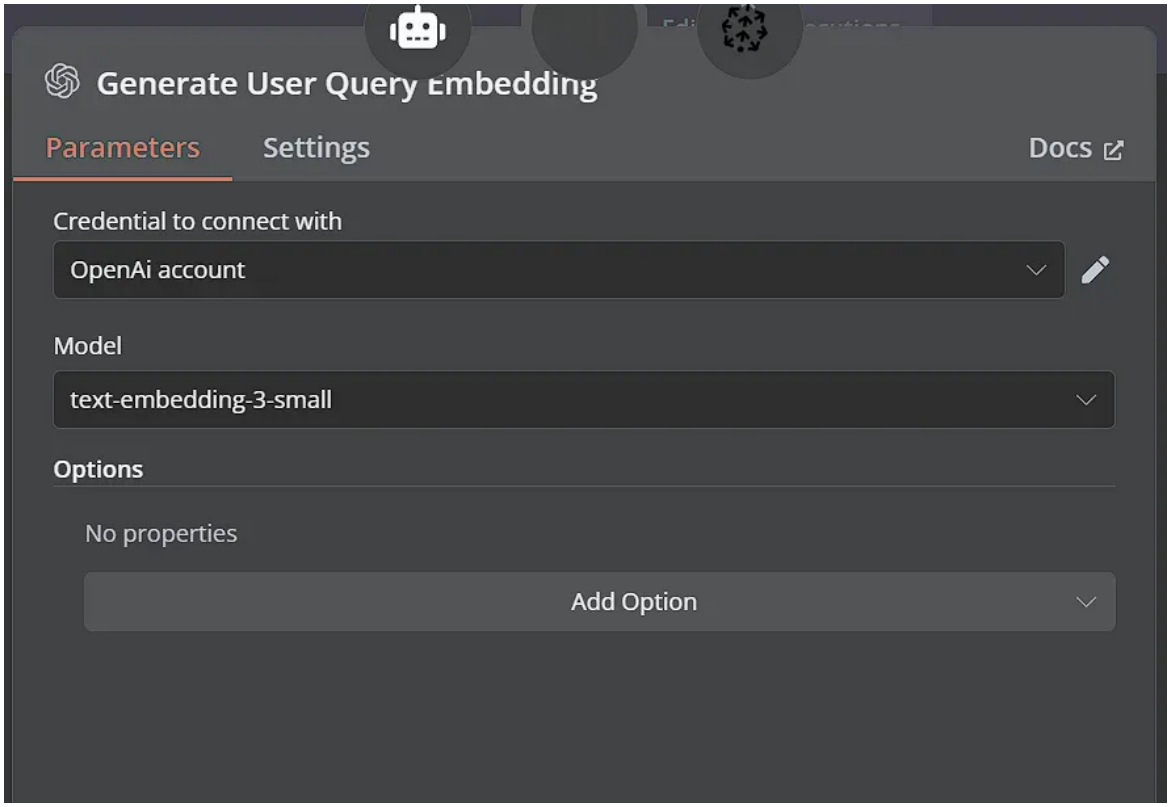
También podemos limitar el número de resultados que recuperamos del almacén de vectores a los 4 más relevantes para la consulta del usuario.



Luego, conecte el nodo Pinecone Vector Store . Esta vez, configure el **modo de operación** como *"Recuperar documentos (para agente/cadena)"* y conecte el mismo nodo Embeddings OpenAI que usamos anteriormente, asegurándose de seleccionar el mismo modelo text-embedding-3-large. Esta configuración generará incrustaciones para la consulta del usuario, lo que permitirá compararlas con todas las incrustaciones en su almacén vectorial.



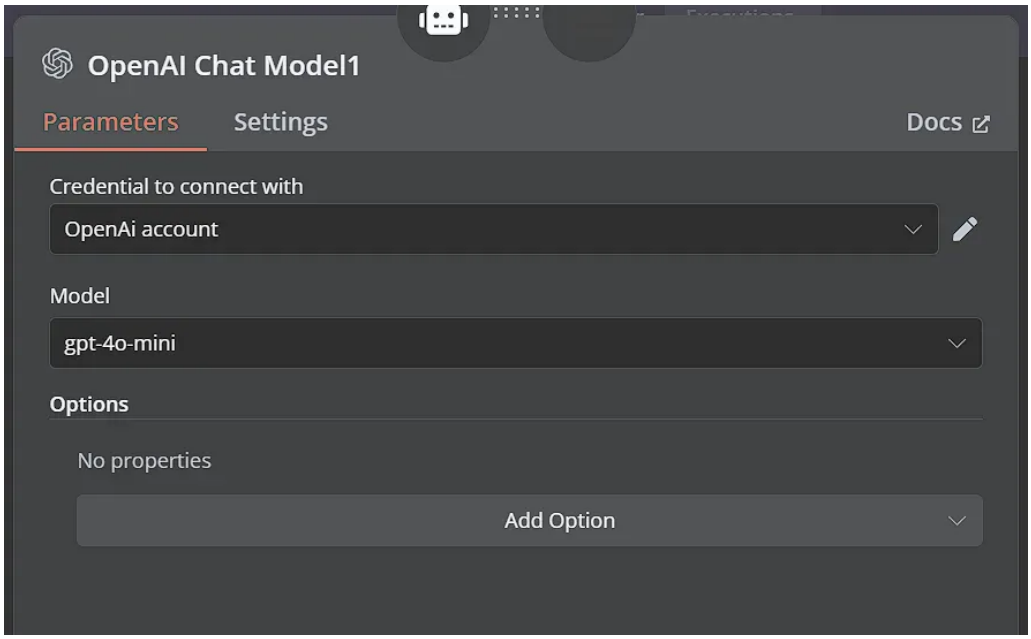
Recuperar documentos de la tienda de vectores Pinecone



Generar incrustaciones de consultas de usuario

Finalmente, conecte otro nodo del Modelo de Chat de OpenAI . Este nodo resumirá los fragmentos recuperados de la base de datos, proporcionando contexto para la respuesta final.

También puedes utilizar aquí el mismo modelo gpt-4o-mini.



Configurar el modelo de chat OpenAI gpt-4o-mini

Paso 6: Prueba tu chatbot RAG

¡Y ahí lo tienes!

Ahora ha configurado los componentes principales de su chatbot de API de GitHub impulsado por RAG.

Para probarlo, simplemente haz clic en el botón "Chat" ubicado en la parte inferior del editor n8n. Esto abrirá la interfaz de chat donde podrás empezar a interactuar con tu bot. Como pregunta de prueba, intenta preguntar: "¿Cómo creo una aplicación de GitHub a partir de un manifiesto?". Te sorprenderá ver cómo el chatbot recupera información relevante de la especificación de la API de GitHub y te proporciona una solución detallada, paso a paso, usando la API. Esto demuestra el poder de RAG y cómo n8n facilita la creación de aplicaciones sofisticadas basadas en IA.

Chat

Session de9e5b3211b94f7cb248a7026e3e1100

How do I create a GitHub App from a manifest?

To create a GitHub App from a manifest, follow these steps:

1. **Receive the Code**: When starting the creation process, you will receive a temporary `code`.
2. **Make a POST Request**:
   - Use the GitHub API endpoint: `/app-manifests/{code}/conversions`.
   - Replace `{code}` in the URL with the temporary code you received.
3. **Parameters**: You need to include the `code` parameter in the path of your request.
4. **Response**: A successful request will return a response that includes the GitHub App's `id`, `pem` (private key), and `webhook_secret`.

For more details, you can refer to the API method documentation.

Type a message, or press 'up' arrow for previous one